

MODULE 5 · WEEK 1

ML on Text Data

Turn raw text into numbers a model can learn from: cleaning, bag-of-words, TF-IDF, and classifying real weather reports.

Lectures 49–56



This week

- 1 **Text as data** — why most data is unstructured text
- 2 **Preprocessing** — lowercase, strip, tokenize, word clouds
- 3 **Bag-of-words** — unigrams, bigrams, trigrams
- 4 **TF-IDF** — weighting words by how informative they are
- 5 **Modeling** — classify hail vs. flash-flood reports

Why text analytics

- Most of the world's data is unstructured text — emails, reviews, reports, notes, posts.
- Computers can't read; we convert words into a numeric feature matrix they can learn from.
- **TF-IDF** is the workhorse: fast, interpretable, and still useful for many real problems.

Key idea The whole game is turning words into numbers without losing the signal.

Preprocessing the text

- **Lowercase** and **strip punctuation / funky characters** so "Hail" = "hail".
- Remove **stop words** (the, and, of) that carry no predictive signal.
- **Tokenize** into words, and use a **word cloud** to eyeball what's common.

```
text = re.sub(r'[^a-z ]', '', text.lower())
tokens = [w for w in text.split() if w not in stopwords]
```



Prepping the weather dataset

- The dataset: thousands of National Weather Service storm reports with a text narrative.
- Real data is messy — handle **missing / dirty** values before modeling.
- EDA and (where relevant) resampling to consistent time horizons set up a clean problem.

Bag-of-words: unigrams, bigrams, trigrams

- **Bag-of-words**: a column per word, 1/0 (or a count) for whether it appears.
- **n-grams** capture short phrases: "flash" + "flood" → "flash flood".
- More n-grams = more features — powerful, but watch out for too many columns.

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer(ngram_range=(1, 2))
X = cv.fit_transform(corpus)
```

TF-IDF

- **TF-IDF** weights a word by how often it appears here vs. how rare it is everywhere.
- Common-everywhere words get downweighted; distinctive words get boosted.
- Often a small but real improvement over plain counts — and just as easy to use.

```
from sklearn.feature_extraction.text import TfidfVectorizer  
tfidf = TfidfVectorizer(ngram_range=(1, 2))  
X = tfidf.fit_transform(corpus)
```

Key idea TF-IDF rewards words that are common in a document but rare across the corpus — i.e. informative ones.



Train & interpret the classifier

- Feed the TF-IDF matrix into a standard classifier and read the classification report.
- Inspect the most predictive words/n-grams per class — interpretable by design.
- Bag-of-words/TF-IDF ignore word order; next week's transformers add context.



Lab this week

- Clean and vectorize text with TF-IDF, train a classifier, and evaluate with a classification report.
- Interpret which words/n-grams are most predictive for each class.

In the labs repo: `Module5/Week1_TextAnalytics/text_classification.py`



Key takeaways

- Most data is text — the job is turning words into a numeric matrix.
- Preprocess: lowercase, strip, remove stop words, tokenize.
- Bag-of-words + n-grams capture words and short phrases.
- TF-IDF weights words by how informative they are.
- Classify and interpret — TF-IDF models are fast and explainable.